SafeGuardian: A Web-Based Emergency SOS and Email Notification System

A Mini Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

K. Vamsidhar – 2420030341(Team Lead)

K.V.S Jaswanth - 2420030467

B. Akhil Varma - 2420030141

Under the Esteemed Guidance of

Dr. C. Srikanth

SOA Section – S10



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled "**SafeGuardian: A Web-Based Emergency SOS and Email Notification System** "is a record of Bonafide work of **K. Vamsidhar - 2420030341, K.V.S Jaswanth – 2420030467, B. Akhil Varma – 2420030141.** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

K. Vamsidhar – 2420030341

K.V.S Jaswanth – 2420030467

B. Akhil Varma – 2420030141

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled "**SafeGuardian: A Web-Based Emergency SOS and Email Notification System"** is a bonafide work done and submitted by **K. Vamsidhar – 2420030341, K.V.S. Jaswanth – 2420030467, B. Akhil Varma – Roll. 2420030141** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Course Co-ordinator **Signature of HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Dr. C. Srikanth of SOA Section S10**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwara Rao Sir (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

K. Vamsidhar – 2420030341

K.V.S Jaswanth – 2420030467

B. Akhil Varma – 2420030141

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	20
8.	Conclusion & Future Scope	25
9.	References	27

1. ABSTRACT

SafeGuardian is a web-based women's safety application developed to help users send emergency alerts to their registered contacts. The system provides authenticated access, emergency contact management, location sharing, and incident tracking through a single web interface.

The frontend is developed using HTML, CSS, and JavaScript, while Node.js and Express handle the backend operations. PostgreSQL is used to store user information, emergency contacts, SOS incidents, and notification records. When an authenticated user activates the SOS button, the application obtains the available browser location and creates an incident record. It then generates individual notification records for the user's registered emergency contacts. A separate notification worker processes these records and sends email alerts through an SMTP service. The system also allows users to resolve active incidents and view their previous SOS history.

The project was evaluated through functional checks covering authentication, location display, contact management, SOS activation, email notifications, incident resolution, and history retrieval. The tested core workflow was completed successfully in the controlled validation environment. SafeGuardian demonstrates how a web application can combine emergency incident management with asynchronous email notification processing to support an emergency alert workflow.

Team Name: SafeGuardian Team

Team Logo (if any): N/A

Team Members:

1. **K. Vamsidhar – 2420030341**
   Role: Team Lead
2. **K.V.S. Jaswanth – 2420030467**
   Role: Team Member
3. **B. Akhil Varma – 2420030141**
   Role: Team Member

2. INTRODUCTION

Women's safety is an important concern in today's society. In emergency situations, getting help quickly and sharing information with trusted people can be difficult. Technology can help address this challenge by providing applications that allow users to send emergency alerts and share their current location with registered contacts.

SafeGuardian is a web-based women's safety application developed to provide a simple way for users to activate an emergency SOS alert. The main purpose of this project is to help users communicate emergency information to their registered contacts while maintaining a record of each incident. The system focuses on emergency contact management, location sharing, incident tracking, and email-based notifications.

The application is developed using HTML, CSS, and JavaScript for the frontend, Node.js and Express for the backend, and PostgreSQL for database management. Users can log in, manage their emergency contacts, and activate the SOS button through the dashboard. When an authenticated user activates SOS, the application obtains the available location through the browser's Geolocation API and sends the incident information to the backend. The backend validates the request, stores the incident, and creates notification records for the registered emergency contacts.

One of the important aspects of SafeGuardian is its separate notification worker. Instead of making the initial SOS request wait for the complete email-delivery process, the application stores pending notification records in the database. The independent worker processes these records and sends email alerts through an SMTP service. The system also allows users to resolve active incidents through password confirmation and view their previous incident history.

The project demonstrates a lightweight service-oriented design with application-level workflow coordination and asynchronous email notification processing. It focuses on separating emergency incident management from external email delivery while maintaining records that can be inspected later.

This report presents the literature survey, problem statement, objectives, hardware and software requirements, implementation details, and results of the SafeGuardian application. It also discusses the project's conclusion, limitations, and possible future enhancements.

3. LITERATURE SURVEY

1. Rakshak: A Multi-Layered Intelligent Framework for Offline-Capable Proactive Emergency Response in Women's Safety Systems (2026) : This research presents a multi-layered framework designed to improve women's safety through emergency-response mechanisms. It explores alternative ways of initiating emergency alerts, including approaches that can work when normal interaction is difficult. The study provides a reference for designing emergency-response workflows. SafeGuardian follows a different approach by focusing on user-activated SOS incidents and asynchronous email notifications.

2. Women SOS Safety App (2024) : This project focuses on providing emergency communication and GPS-based location information to predefined contacts. It demonstrates how a mobile application can help users communicate during emergency situations. The work is relevant to SafeGuardian because both systems provide an application-based emergency alert mechanism. SafeGuardian additionally maintains persistent incident records and individual notification records for registered contacts.

3. Women Safety System Using IoT (2022) : This research explores the use of Internet of Things technology in women's safety applications. It considers connected technologies as a way to support emergency communication and safety-related functions. IoT-based approaches can help connect devices and communication systems for alert generation. SafeGuardian differs by using a browser-based interface instead of dedicated IoT hardware.

4. Enhanced Real-Time Women Safety App: Alert Notification and Emergency Response System With Live Location, Video, and Audio Streaming (2025) : This work explores an emergency application that combines alert notifications with live location sharing, video, and audio streaming. These features can provide additional information during an emergency. The approach demonstrates how multimedia communication can support emergency response. SafeGuardian has a narrower scope, concentrating on browser-based location sharing and email notifications.

5. An SOS Women Safety Device: Automatic Emergency Alerts and Geolocation Sharing with GSM/GPS Integration (2026) : This research focuses on a safety device that uses GSM and GPS technologies to support automatic emergency alerts and location sharing. It demonstrates the use of hardware and communication modules for transmitting emergency information. Such systems provide an alternative to applications that require manual interaction through a browser or mobile interface. SafeGuardian instead uses explicit user activation and browser geolocation.

6. Cross-Platform Women Safety Mobile Application with Real-Time Alert System and GPS Tracking (2026) : This research presents a cross-platform mobile application designed to support women's safety through real-time alerts and GPS tracking. It focuses on making emergency communication and location information available through a mobile application. The study is relevant to SafeGuardian because both approaches use

location information as part of emergency communication. SafeGuardian implements these functions through a web-based application and email notification workflow.

7. Thozha: A Smart Mobile and IoT-Based Safety Ecosystem for Women (2025) : The Thozha system explores a combination of mobile application technologies and Internet of Things features for women's safety. It represents an integrated approach in which software and connected devices contribute to emergency assistance. The work highlights the different technologies that can be combined to support safety applications. SafeGuardian focuses on a simpler web-based architecture with database-backed incident management and asynchronous email processing.

8. The Role of IoT in Woman's Safety: A Systematic Literature Review (2023) : This study reviews the role of Internet of Things technologies in women's safety. It examines different safety devices, sensors, communication technologies, and their possible applications. The review provides an overview of existing approaches and the design considerations involved in developing IoT-based safety solutions. It helps identify the differences between hardware-oriented systems and software-based emergency applications such as SafeGuardian.

9. An Integrated Mobile Applications for Enhancing Women's Safety – A Comprehensive Approach (2024) : This research discusses an integrated mobile application approach for enhancing women's safety. It considers coordination between applications designed for women and police, along with location-aware emergency information and computer-aided dispatch capabilities. The approach extends emergency communication beyond personal contacts. SafeGuardian currently focuses on notifying registered emergency contacts and does not include police dispatch integration.

10. Smart Women Safety Device Using IoT and GPS Tracker (2023) : This research explores a women's safety device that combines IoT technology and GPS tracking. It focuses on using connected hardware to support location tracking and emergency communication. The work demonstrates how GPS and IoT components can be applied in safety-related systems. SafeGuardian uses browser geolocation and a web-based SOS workflow instead of a dedicated tracking device.

4. Problem Statement and Objectives

Problem Statement:

In emergency situations, individuals may face difficulties in quickly alerting trusted contacts and sharing their current location for timely assistance.

Problem Description:

Existing communication methods may require users to contact people individually during emergencies. SafeGuardian addresses this challenge by providing an SOS feature that records emergency incidents and sends email alerts with available location information to registered emergency contacts.

Objectives:

1. Provide Emergency SOS Functionality:
Enable authenticated users to activate an SOS alert through a simple interface.

2. Share Location Information:
Obtain the user's available location through the browser's Geolocation API and include it in the emergency alert.

3. Manage Emergency Contacts:
Allow users to add, view, and manage their registered emergency contacts.

4. Implement Email Notifications:
Send emergency email alerts to registered contacts using an asynchronous notification worker.

5. Maintain Incident Records:
Store SOS incidents and notification statuses in PostgreSQL for tracking and future reference.

6. Support Incident Resolution:
Allow users to resolve active SOS incidents through password confirmation and view their previous SOS history.

7. Separate Notification Processing:
Keep email delivery separate from the main SOS request so that notification processing can happen asynchronously.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- **Processor:** Intel Core i3 or equivalent processor and above.
- **RAM:** Minimum 4 GB RAM.
- **Storage:** Minimum 500 MB of free storage for project files and required software.
- **Internet Connection:** Required for email notifications and location-related services.

Software Requirements:

- **Operating System:** Windows 10/11 or equivalent.
- **Frontend Technologies:** HTML, CSS, and JavaScript.
- **Backend Technologies:** Node.js and Express.js.
- **Database:** PostgreSQL.
- **Development Tools:** Visual Studio Code and a web browser.
- **Libraries and Services:** Node.js pg, Express Session, bcrypt, Nodemailer, and SMTP service.
- **Browser API:** Geolocation API for obtaining the user's available location.

6. IMPLEMENTATION

1. Frontend:

The frontend of SafeGuardian is developed using HTML, CSS, and JavaScript. It provides a user-friendly interface for registration, login, dashboard access, emergency contact management, SOS activation, location display, and incident history.

HTML is used to structure the web pages, CSS is used for styling and layout, and JavaScript handles user interactions, browser geolocation, and communication with the backend through REST API requests.

Frontend files and their purpose:

- **HTML files:** Define the structure of login, registration, dashboard, and other application pages.
- **CSS files:** Manage page styling, layout, buttons, forms, and responsive appearance.
- **JavaScript files:** Handle authentication-related interactions, emergency contact operations, location acquisition, SOS activation, and API requests.

2. Backend:

The backend is developed using Node.js and Express.js. It handles user authentication, session management, emergency contact operations, SOS incident creation, incident resolution, and history retrieval.

PostgreSQL is used to store user details, emergency contacts, SOS incidents, and notification records. Passwords are protected using bcrypt, and Express Session is used to maintain authenticated user sessions.

Backend files and their purpose:

- **Route files:** Handle REST API endpoints for user authentication, emergency contacts, and SOS operations.
- **Authentication middleware:** Checks user sessions before allowing access to protected operations.
- **Database integration files:** Establish PostgreSQL connectivity and execute database queries.
- **Notification service:** Creates and manages email notification records.
- **Notification worker:** Independently processes pending notification records and sends emails through SMTP.

3. API Integration:

SafeGuardian uses REST APIs to connect the frontend with the backend. The frontend sends HTTP requests to the Express server for registration, login, emergency contact management, SOS activation, incident resolution, and history retrieval.

The Browser Geolocation API obtains the user's location when permission is granted and the device supports it. The backend stores the SOS incident and creates notification records in PostgreSQL. An independent notification worker retrieves pending records and sends emergency email alerts using Nodemailer and an SMTP service.

Integration components:

- REST API endpoints for frontend and backend communication.
- PostgreSQL integration using the Node.js pg library.
- Express Session and bcrypt for authentication.
- Browser Geolocation API for location acquisition.
- Nodemailer and SMTP for email notifications.
- Independent Node.js worker for asynchronous notification processing.

6.1 Frontend (HTML, CSS and JavaScript)

The frontend provides the login interface, dashboard, emergency contact management, location display, and SOS activation.

Important frontend files:

- public/index.html – Login page.
- public/register.html – User registration page.
- public/dashboard.html – Main dashboard and SOS interface.
- public/css/style.css – Login and registration page styling.
- public/css/dashboard.css – Dashboard styling.
- public/js/login.js – Login form handling.
- public/js/register.js – Registration form handling.
- public/js/dashboard.js – Location acquisition, contact management, SOS activation, and history operations.

Code 1: SOS Button – public/dashboard.html

```
<button id="sosButton" class="sos-button">
  SOS
</button>

<p id="sosStatus">
  No active emergency
</p>
```

Explanation: Creates the SOS button and displays the current emergency status on the dashboard.

Code 2: Browser Location – public/js/dashboard.js

```
function getCurrentLocation() {
  if (!navigator.geolocation) {
    return;
  }

  navigator.geolocation.getCurrentPosition(
    position => {
      const latitude = position.coords.latitude;
      const longitude = position.coords.longitude;

      currentLocation = {
        latitude,
        longitude
      };
    },
    error => {
      console.error("Location error:", error);
    }
```

```
    );
}
```

Explanation: Uses the browser Geolocation API to obtain the user's latitude and longitude when permission is granted.

Code 3: Three-Second SOS Activation – public/js/dashboard.js

```
function startSosHold(event) {
    event.preventDefault();

    if (!currentLocation || holdStarted) {
        return;
    }

    holdStarted = true;

    holdTimer = setTimeout(() => {
        triggerSos();
    }, 3000);
}
```

Explanation: Activates the SOS request after the user holds the SOS button for three seconds.

6.2 Backend (Node.js and Express.js)

The backend handles user authentication, session validation, emergency contact operations, SOS incident management, and database operations.

Important backend files:

- server.js – Initializes the Express server and registers API routes.
- config/db.js – Establishes PostgreSQL database connectivity.
- routes/authRoutes.js – Handles user registration and login.
- routes/contactRoutes.js – Manages emergency contacts.
- routes/sosRoutes.js – Handles SOS creation, resolution, and history.

- middleware/authMiddleware.js – Protects authenticated routes.
- workers/notificationWorker.js – Processes email notifications.

Code 1: Password Hashing – routes/authRoutes.js

```
const passwordHash =
   await bcrypt.hash(password, 10);


const result = await pool.query(
   `INSERT INTO users
    (name, email, phone, password_hash)
    VALUES ($1, $2, $3, $4)`,
   [name, email, phone, passwordHash]
);
```

Explanation: Uses bcrypt to hash the user's password before storing it in PostgreSQL.

Code 2: Authentication Middleware – middleware/authMiddleware.js

```
function requireAuth(req, res, next) {
   if (!req.session || !req.session.userId) {
      return res.status(401).json({
         message: "Authentication required"
      });
   }


   req.userId = req.session.userId;
   next();
}


module.exports = requireAuth;
```

Explanation: Checks whether the user has an authenticated session before accessing protected operations.

Code 3: SOS Incident Creation – routes/sosRoutes.js

```
await client.query("BEGIN");

const sosResult = await client.query(
   `INSERT INTO sos_incidents
    (user_id, latitude, longitude, locality,
     status, locked)
    VALUES ($1, $2, $3, $4, 'ACTIVE', TRUE)
    RETURNING id, status`,
   [
      req.userId,
      latitude,
      longitude,
      locality || null
   ]
);

await client.query("COMMIT");
```

Explanation: Creates an active SOS incident in PostgreSQL using a database transaction. The complete route also checks for an existing active incident and creates notification records before committing.

6.3 API Integration

REST APIs connect the frontend with the backend. PostgreSQL stores application records, while the independent notification worker handles email delivery through SMTP.

Important integration files:

- public/js/login.js – Sends login requests to the backend.
- public/js/dashboard.js – Sends SOS and other dashboard requests.
- routes/authRoutes.js – Provides authentication API endpoints.
- routes/sosRoutes.js – Provides SOS API endpoints and creates notification records.

- config/db.js – Connects Node.js with PostgreSQL using the pg library.
- workers/notificationWorker.js – Retrieves pending email notifications and sends emails using Nodemailer.

Code 1: Frontend-to-Backend SOS Request – public/js/dashboard.js

```
const response = await fetch("/api/sos", {
   method: "POST",

   headers: {
      "Content-Type": "application/json"
   },

   credentials: "include",

   body: JSON.stringify({
      latitude: currentLocation.latitude,
      longitude: currentLocation.longitude,
      locality: "Current location"
   })
});

const data = await response.json();
```

Explanation: Sends the user's location to the Express backend through an authenticated REST API request.

Code 2: Retrieving Pending Notifications – workers/notificationWorker.js

```
const result = await pool.query(
   `SELECT n.id, n.message,
         c.name AS contact_name,
         c.email AS contact_email
    FROM notifications n
```

```
    JOIN emergency_contacts c
      ON c.id = n.contact_id
    WHERE n.status = 'PENDING'
      AND n.channel = 'EMAIL'
    ORDER BY n.id
    LIMIT 10`
);
```

Explanation: Retrieves pending email notification records and their associated emergency contact details from PostgreSQL.

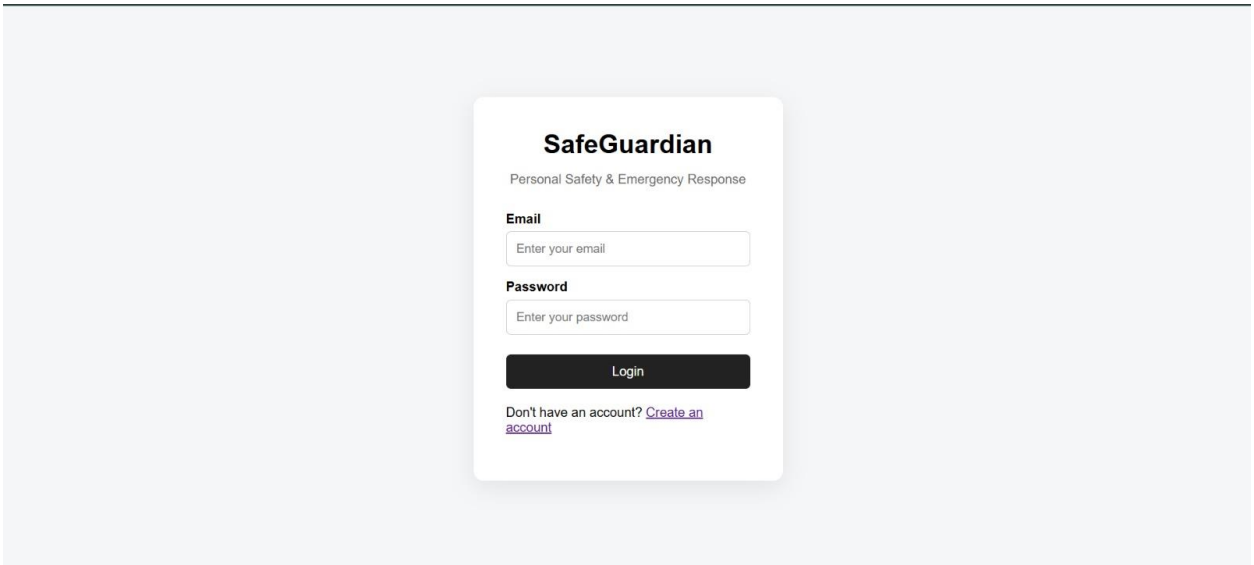
Code 3: Sending Emergency Email – workers/notificationWorker.js

```
await emailTransporter.sendMail({
   from: `"SafeGuardian" <${process.env.EMAIL_FROM}>`,
   to: email,
   subject: "SafeGuardian Emergency Alert",
   text: message
});
```

Explanation: Uses Nodemailer and SMTP to send emergency alert emails to registered contacts. The worker subsequently updates the notification status in PostgreSQL.

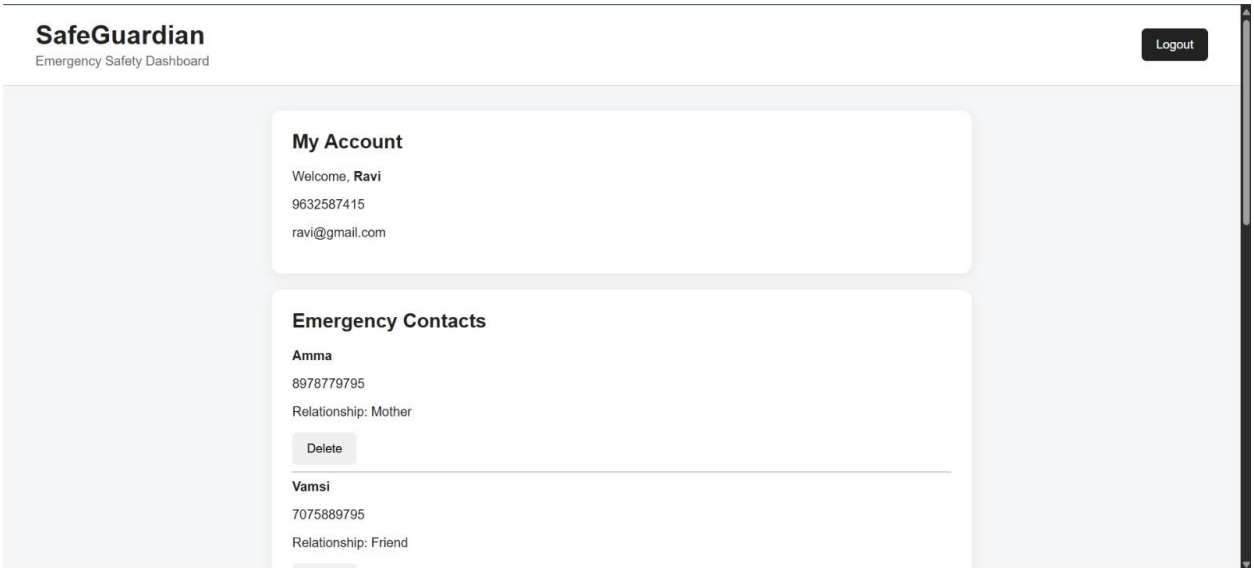
7. RESULTS & DISCUSSION

7.1 Login Page



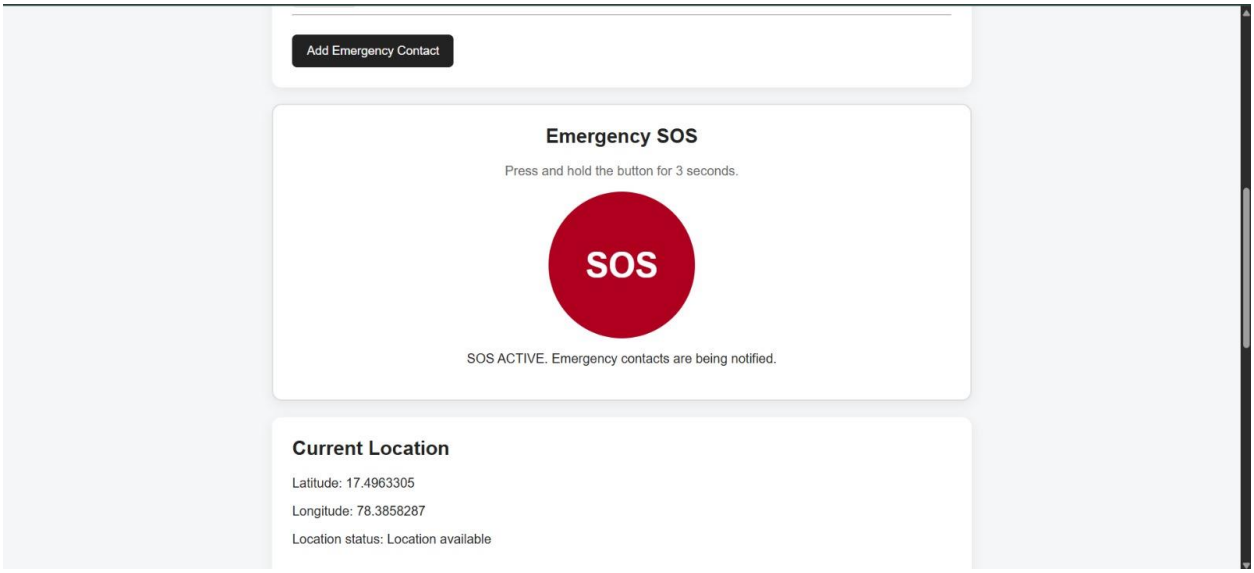
Screenshot 1: User Login Interface
The login page allows registered users to securely access the SafeGuardian application using their credentials. It provides an entry point to the emergency safety features.

7.2 Dashboard and Emergency Contacts



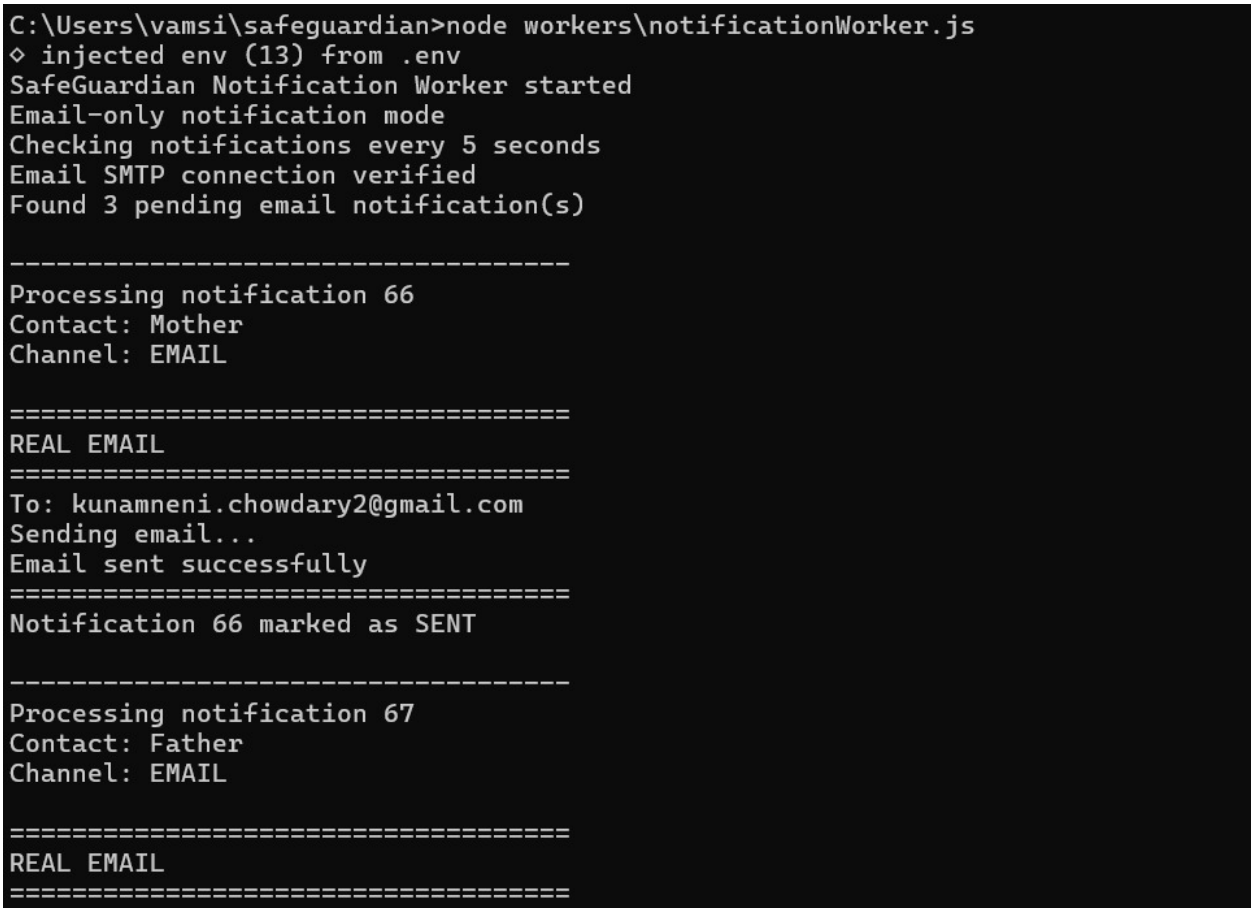
Screenshot 2: User Dashboard and Emergency Contact Management
The dashboard displays the user's emergency information and available application features. Users can manage their registered emergency contacts through this interface.

7.3 SOS Activation and Location



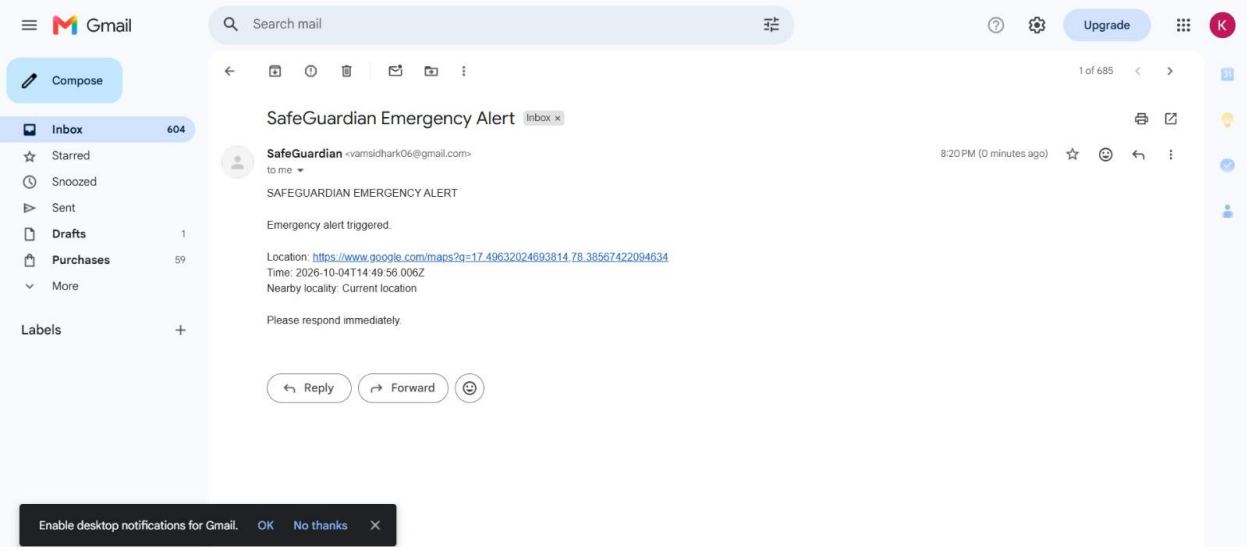
Screenshot 3: SOS Button and Current Location
The user can activate the SOS feature by pressing and holding the button for three seconds. The application obtains the available browser location and submits the emergency request to the backend.

7.4 Notification Worker



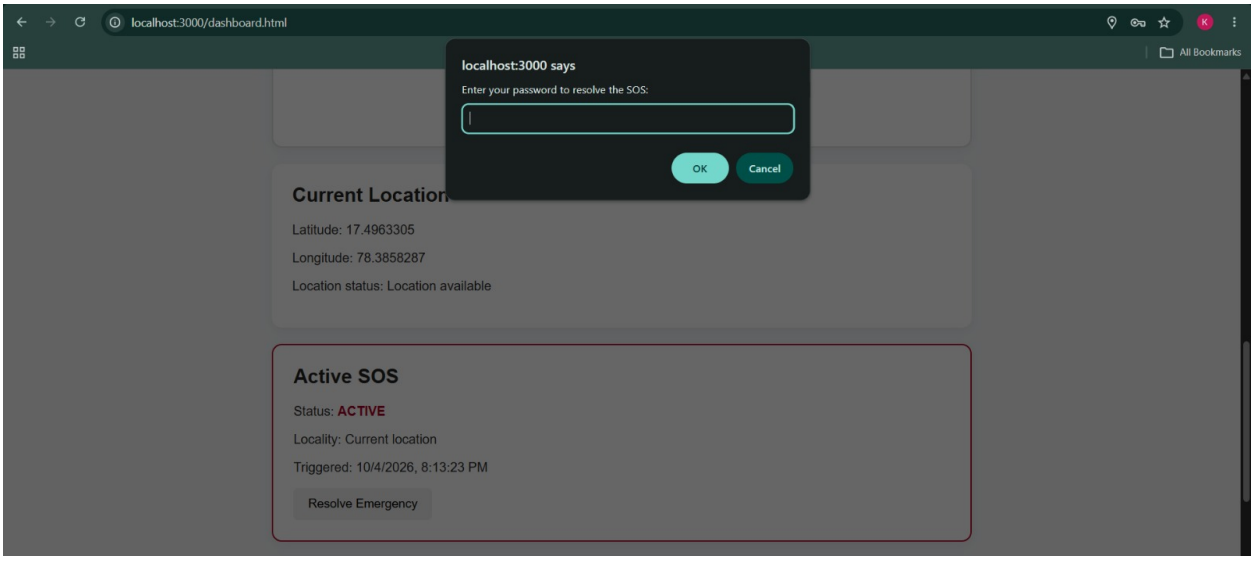
Screenshot 4: Asynchronous Notification Worker Execution
The notification worker retrieves pending email notification records from PostgreSQL. It processes the records independently and attempts to send emergency emails through SMTP.

7.5 Emergency Email Notification



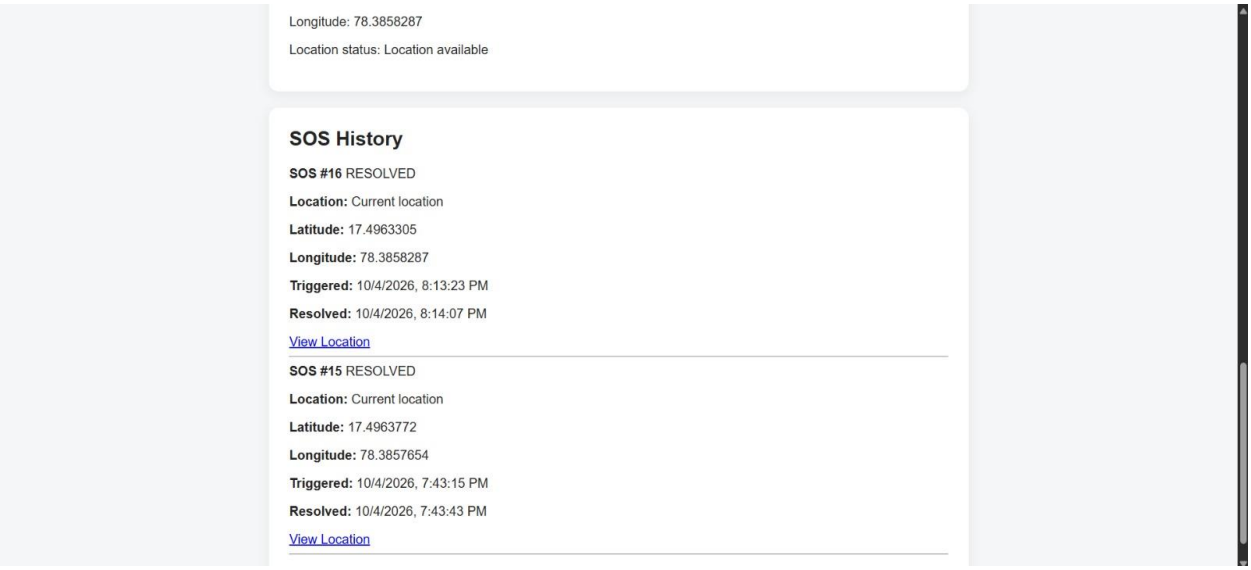
Screenshot 5: Received Emergency Email
The registered emergency contact receives an email containing the emergency alert information. The email includes a Google Maps link to view the available incident location.

7.6 SOS Incident Resolution



Screenshot 6: Password Confirmation for SOS Resolution
The application requires password confirmation before resolving an active SOS incident. After successful verification, the incident status is updated to RESOLVED.

7.7 SOS History



Screenshot 7: Emergency Incident History
The history page displays previously recorded SOS incidents and their respective statuses. It helps users review their emergency activity and incident records.

7.8 Overall Discussion

The functional testing demonstrated the successful operation of the main SafeGuardian workflow, including authentication, emergency contact management, location acquisition, SOS activation, email notification, incident resolution, and history retrieval. The results confirm the functional implementation of the application in the tested environment.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

SafeGuardian was developed as a web-based emergency SOS and email notification system to help users communicate emergency information to their registered contacts. The project successfully integrates user authentication, emergency contact management, browser-based location acquisition, SOS incident creation, and incident tracking.

The application uses HTML, CSS, and JavaScript for the frontend, Node.js and Express for backend operations, and PostgreSQL for data storage. An independent notification worker processes pending notification records and sends emergency email alerts through SMTP. This approach separates incident creation from email delivery and allows notification statuses to be maintained.

The system was functionally tested for login, location display, emergency contact management, SOS activation, email notification, incident resolution, and SOS history retrieval. The tested core workflow operated successfully in the controlled development environment.

Overall, SafeGuardian demonstrates how web technologies, database management, and asynchronous processing can be combined to develop a lightweight emergency alert application. The project objectives were achieved within the scope of the implemented prototype.

8.2 Future Scope

The application can be enhanced in the following stages:

A. Short-Term Enhancements

1. Improve the user interface and make the application more responsive across mobile devices and desktop systems.
2. Improve notification monitoring and provide clearer delivery-status information.
3. Add more detailed incident information and improve SOS history management.

B. Medium-Term Enhancements

1. Integrate additional notification channels such as SMS and messaging platforms.
2. Develop a dedicated mobile application for easier access to emergency features.
3. Introduce trusted-contact acknowledgement and response tracking.

C. Long-Term Enhancements

1. Explore integration with authorized emergency response services.
2. Improve location-sharing capabilities to support continuous location updates during an active emergency.
3. Conduct large-scale testing across different devices, networks, and communication providers to evaluate system reliability and scalability.

These enhancements can extend SafeGuardian into a more comprehensive personal safety platform while retaining its core objective of simplifying emergency communication.

9. REFERENCES

1. **Rakshak: A Multi-Layered Intelligent Framework for Offline-Capable Proactive Emergency Response in Women's Safety Systems (2026).** https://doi.org/10.2991/978-94-6239-713-2_50
2. **Y. Saravanan and S. Marjudi, "Women SOS Safety App" (2024).** https://periodical.uthm.edu.my/index.php/aitcs/article/view/12242
3. **"Women Safety System Using IoT" (2022).** https://doi.org/10.1109/ICAAIC53929.2022.9792929
4. **"Enhanced Real-Time Women Safety App: Alert Notification and Emergency Response System With Live Location, Video, and Audio Streaming" (2025).** https://doi.org/10.1109/ICDSBS63635.2025.11031609
5. **"An SOS Women Safety Device: Automatic Emergency Alerts and Geolocation Sharing with GSM/GPS Integration" (2026).** https://doi.org/10.1109/IC-ICNS68863.2026.11537940
6. **P. Jothilakshmi, M. S. Rishikesh, and M. Y. Srikanth, "Cross-Platform Women Safety Mobile Application with Real-Time Alert System and GPS Tracking" (2026).** https://doi.org/10.1109/ICESIC67389.2026.11496592
7. **"Thozha: A Smart Mobile and IoT-Based Safety Ecosystem for Women" (2025).** https://doi.org/10.1109/ICCDS64403.2025.11209496
8. **M. S. Farooq et al., "The Role of IoT in Woman's Safety: A Systematic Literature Review" (2023).** https://doi.org/10.1109/ACCESS.2023.3252903
9. **K. Ashwini and S. N. Perla, "An Integrated Mobile Applications for Enhancing Women's Safety – A Comprehensive Approach" (2024).** https://doi.org/10.1109/ICWITE59797.2024.10503411
10. **G. Uganya et al., "Smart Women Safety Device Using IoT and GPS Tracker" (2023).** https://doi.org/10.1109/ICCEBS58601.2023.10449302